# Contract testing and Pact interview questions, answered with this project

Questions about contract testing, Pact, the Pact Broker and deployment safety that come up in SDET and test automation interviews. Each answer has a short explanation and, where useful, an example, and ends with an **In the project** pointer to where this repository does it (or why it does not). Links go to github.com/niranjankagri/pact-contract-testing.

Versions: Pact JS 17 (Pact specification V4), Pact JVM 4.7 (JUnit 5), Pact Broker 2.121, pact-broker CLI 18, Spring Boot 4.1, Java 17, Node 24.

## Contents


1. Contract testing basics (Q1-Q10)
2. Pact concepts (Q11-Q22)
3. The consumer side (Q23-Q32)
4. The provider side (Q33-Q44)
5. Pact Broker and deployment safety (Q45-Q60)
6. CI/CD and team process (Q61-Q67)
7. Scenarios and pitfalls (Q68-Q76)
8. Quick reference


---

## 1. Contract testing basics

### Q1. What is contract testing?

A way to check that two services that talk to each other agree on the messages they exchange, **without running them together**. Each side is tested alone against a shared description of the interactions (the contract). If both sides pass, they work together, as far as the contract goes.

**In the project:** checkout-web (consumer) and product-service (provider) are tested separately; the pact file connects them.

### Q2. Why not just end-to-end tests?

End-to-end tests need every service deployed in one environment at compatible versions: slow to set up, slow to run, flaky, and they find problems late. Their failures are hard to locate ("which service broke it?"). Contract tests run in each service's own build, in seconds, and point at the exact interaction and field that broke. End-to-end tests are still useful for a few critical journeys.

**In the project:** the full contract workflow runs in about 35 seconds locally; no environment with both services is needed.

### Q3. Where do contract tests sit in the test pyramid?

Between unit/component tests and end-to-end tests: they are as fast and isolated as unit tests but check an integration point. They replace many integration tests between services, not the unit tests inside a service.

**In the project:** each side also has its own unit or HTTP tests ( `checkout.test.ts` , `ProductApiTest` ).

### Q4. What does "consumer-driven" mean?

The consumer writes the contract from its real needs: only the requests it sends and the fields it reads. The provider then has to satisfy every consumer's contract. Benefit: the provider learns exactly what is used, and may change everything else freely.

**In the project:** the provider sends `description` , `category` , `createdAt` ; the contract does not contain them, because checkout-web does not use them.

### Q5. Consumer-driven vs provider-driven contracts?

Provider-driven (for example Spring Cloud Contract, or an OpenAPI spec): the provider publishes what it offers, consumers test against stubs of it. Consumer-driven (Pact): each consumer states what it needs. Provider-driven fits public APIs with unknown consumers; consumer-driven fits internal services with known consumers.

**In the project:** consumer-driven with Pact; design doc section 7 mentions bi-directional contracts.

### Q6. Is contract testing the same as schema validation?

No. A schema (OpenAPI, JSON Schema) says what a provider **may** send. A contract says what a consumer **does** send and **needs** back, per interaction, including states ("given this product exists"), status codes and error bodies. A response can match the schema and still break a consumer (a required field made optional, an enum value the consumer does not handle).

**In the project:** the 409 contract requires `available` , which a generic schema might list as optional.

### Q7. What does a contract test NOT check?

Business logic of the provider (is the price correct?), performance, security beyond what is in the request, and behaviour that no consumer uses. Those belong to the provider's own functional tests.

**In the project:** `ProductApiTest` and `ReservationServiceTest` test the provider's rules; the contract only the shape and status of what checkout-web uses.

### Q8. When is contract testing worth it?

When several teams own services that call each other and deploy independently (microservices, mobile/web front ends with a BFF). Less useful for a monolith, a single team deploying everything together, or a public API with unknown consumers.

**In the project:** two "teams" in two languages, deploying separately (each has its own `can-i-deploy` ).

### Q9. Which tools exist?

Pact (many languages, consumer-driven, with a broker), Spring Cloud Contract (JVM, provider-driven, Groovy/YAML contracts), PactFlow (hosted broker, bi-directional contracts from OpenAPI), and schema-based checks such as Specmatic or Dredd for OpenAPI.

**In the project:** Pact JS + Pact JVM + the open-source Pact Broker.

### Q10. How do you introduce contract testing in a company?

Start with one important integration with a cooperative provider team, write consumer tests for the interactions used, get the provider to verify them in its build, add a broker, then `can-i-deploy` in the deployment pipelines. Measure: fewer integration failures in shared environments, fewer end-to-end tests.

**In the project:** shows the end state of that path for one integration.

---

## 2. Pact concepts

### Q11. What are the main Pact terms?

- **Consumer / provider**: the client and the service it calls ("pacticipants").
- **Interaction**: one request and its expected response, with a description and optional provider states.
- **Pact file**: the contract, a JSON file with all interactions between one consumer and one provider.
- **Mock server**: Pact's stand-in for the provider during consumer tests.
- **Verification**: replaying the pact against the real provider.
- **Pact Broker**: the server that stores pacts and verification results.

**In the project:** `pacts/checkout-web-product-service.json` with 6 interactions.

### Q12. What is in a pact file?

Consumer and provider names, the interactions (description, provider states with parameters, request: method, path, query, headers, body; response: status, headers, body), the matching rules for each field, and metadata such as the Pact specification version.

**In the project:** generated in `pacts/`, uploaded as a CI artifact; spec version 4.0.

### Q13. What are provider states?

Preconditions an interaction needs on the provider side, written by the consumer as a name (`'a product exists'`), optionally with parameters. During verification the provider runs a matching state handler that prepares the data before the request is replayed.

**In the project:** `given('a product exists', { id: 1, price: 19.99, stock: 25 })` and the `@State` methods in `ProductPactVerificationBase.java`.

### Q14. Why give provider states parameters?

A generic state with data ("a product exists" with id, price and stock) can be reused by many interactions; the provider has a few handlers instead of one per example. The data travels in the contract, so the provider does not need to guess it.

**In the project:** one state "a product exists" serves four interactions with different products and stock.

### Q15. What are matchers, and why use them?

Matchers relax exact matching: `integer(25)` means "any integer, for example 25"; `regex(/^[A-Z]{3}$/, 'EUR')` means "three capital letters"; `eachLike(x)` means "an array of elements shaped like x". Without matchers the provider would have to return exactly the consumer's example data. Use exact values only where the value matters (an enum the consumer switches on).

**In the project:** `productShape()` in `productService.pact.test.ts`.

### Q16. Which matchers do you use most?

Type matchers (`like`, `integer`, `decimal`, `string`, `boolean`), `eachLike` / `atLeastOneLike` for arrays, `regex` for formats, date/time and `uuid` matchers, and `fromProviderState` for values only the provider knows (generated ids in URLs).

**In the project:** `integer`, `decimal`, `string`, `regex` (currency, ISO instant, Bearer header), `uuid`, `eachLike`.

### Q17. What changed between the Pact specification versions?

V2: type and regex matchers. V3: provider states with parameters and several states, generators, array matchers like `eachKeyLike`, message pacts. V4: HTTP and message interactions in one file, synchronous messages, plugins (gRPC, Protobuf, CSV), comments and per-interaction "pending". Both sides must support the spec version used.

**In the project:** V4 (`PactV4` in Pact JS; Pact JVM 4.7 verifies it).

### Q18. How does the Pact mock server work?

Each consumer test registers its interactions with a local mock server (the Rust core of Pact). The code under test calls the mock server; it answers with the example response if the request matches the expectation, and fails the test if a request is unexpected or an expected request never came. After a passing test the interactions are written to the pact file.

**In the project:** `executeTest(async (mockServer) => new ProductClient(mockServer.url, ...))`.

### Q19. What happens during provider verification?

For each interaction: run the provider-state handlers, send the recorded request (after request filters) to the running provider, then compare status, headers and body with the expected response using the matching rules. Extra fields in the provider's response are allowed; missing or wrongly typed ones fail.

**In the project:** `ProductPactVerificationBase.verifyInteraction` with JUnit 5's `@TestTemplate`.

### Q20. Why are extra response fields allowed but extra request fields not?

Postel's law, applied to JSON: the consumer must ignore fields it does not know (tolerant reader), so the provider may add fields. The provider, however, may reject unknown request fields, so the consumer must send exactly what the contract says.

**In the project:** the provider returns more fields than the contract lists; the verification passes.

### Q21. What are message pacts?

Contracts for asynchronous messages (events on Kafka, queues): the consumer describes the message it can handle, the provider proves it produces such a message. No HTTP involved.

**In the project:** not included (listed under limitations).

### Q22. What is a Pact plugin?

An extension of the Pact core (V4) for other protocols or content types: gRPC and Protobuf, CSV, Avro. The plugin knows how to match and generate that content.

**In the project:** not needed (JSON over HTTP).

---

## 3. The consumer side

### Q23. What should a consumer contract test call?

The real client code (the class that makes HTTP calls), pointed at the mock server. Then the contract records what the code really sends. Calling `fetch` directly in the test would only test the test.

**In the project:** every main interaction uses `ProductClient`; only the feature-branch demo calls `fetch` directly.

### Q24. What should a consumer contract test assert?

That the client handles the response: it returns the right object, throws the right error for 404/409/401. Not the business logic on top: that belongs in unit tests with a fake client.

**In the project:** the pact tests check `ProductClient`; `checkout.test.ts` checks `priceBasket` with a fake.

### Q25. How much of the response should the contract contain?

Only the fields the consumer uses, with matchers. Every extra field is a constraint on the provider that nobody needs, and makes the provider team's changes harder.

**In the project:** `productShape()` has four fields; the provider sends seven.

### Q26. How do you handle authentication in consumer tests?

Put the Authorization header in the contract with a regex matcher (any Bearer token) and an example value. The consumer cannot know a valid provider token; the provider replaces it during verification.

**In the project:** `AUTH_HEADER` in `pactSetup.ts`.

### Q27. Should error responses be in the contract?

Yes, when the consumer has code for them. A 404 or 409 body is part of the API; if the provider changed the error format, the consumer's error handling would break.

**In the project:** 404 `{error}`, 409 `{error, available}`, 401 `{error}`.

### Q28. How do you write a contract for a list?

`eachLike(template)` (at least one element of that shape) or `atLeastLike(template, n)`. Do not depend on an exact length or order unless the consumer really does.

**In the project:** `GET /products?ids=1,2` expects `{ products: eachLike(productShape(...)) }`.

### Q29. How do you handle dates and generated ids?

Format matchers: an ISO-8601 regex or datetime matcher, `uuid()`. If a generated id is part of a later request URL, use `fromProviderState` so the provider can inject its real id.

**In the project:** `expiresAt: isoInstant(...)` accepts Java's optional fraction of a second; `reservationId: uuid()`.

### Q30. Where does the pact file go, and should it be committed?

It is a build output: generated by the consumer tests and published to the broker. Committing it invites stale copies. A local folder is fine for a quick provider check without a broker.

**In the project:** `pacts/` is in `.gitignore`; CI uploads it as an artifact; the provider's folder test reads it locally.

### Q31. Why delete the pact file before each consumer run?

Pact merges interactions into an existing file. A test that was removed would otherwise leave its interaction in the contract, and the provider would keep verifying something nobody needs.

**In the project:** `test/pact/globalSetup.ts`.

### Q32. Why run the Pact test files one after another?

They write to the same pact file; parallel writers can collide. One mock server per test is fine, but files should not run in parallel processes.

**In the project:** `fileParallelism: false` in `vitest.config.ts`.

---

## 4. The provider side

### Q33. How do you set up provider verification in Java?

Pact JVM's JUnit 5 module: a test class with `@Provider("name")`, a source (`@PactFolder` or `@PactBroker`), a `@TestTemplate` method extended with `PactVerificationInvocationContextProvider` that calls `context.verifyInteraction()`, a target (`HttpTestTarget`) and `@State` methods.

**In the project:** `ProductPactVerificationBase.java` + two subclasses.

### Q34. Should the provider run for real during verification?

Yes, as far as possible: real controllers, filters, validation and serialisation. Mock only what is outside the service (other services, slow infrastructure). Mocking the provider's own logic would verify the mocks.

**In the project:** `@SpringBootTest(webEnvironment = RANDOM_PORT)`: the whole application on a free port.

### Q35. How do you implement provider states?

A method per state name that prepares the data, reading the parameters from the contract. Reset first, so every interaction starts clean. For a database: insert rows (or use the service layer); for downstream services: stub them.

**In the project:** `productExists(Map<String, Object> params)` deletes all products and saves exactly the one described.

### Q36. What is a request filter, and when do you need one?

Code that changes each request before it is sent to the provider, for things the consumer cannot know: a valid token, a signature, a tenant header. Change only what is needed, and only where it applies.

**In the project:** replaces the Authorization header only if present, so the "without a token" interaction still gets its 401.

### Q37. What are consumer version selectors?

Rules that tell the broker which consumer versions' pacts to verify: for example the latest on each consumer's main branch, everything deployed or released, and the branch with the same name as the provider's branch.

**In the project:** `mainBranch().deployedOrReleased().matchingBranch()` in `ProductPactBrokerTest`.

### Q38. What are pending pacts?

A pact the provider has never passed is "pending" for that provider branch: its failure is reported but does not fail the provider's build. Once it passes, it is no longer pending, and later failures do fail the build. This stops a consumer's new expectation from blocking the provider team.

**In the project:** `enablePendingPacts = "true"`; the breaking demo shows a failed pending verification with a green build.

### Q39. What are WIP pacts?

"Work in progress" pacts: contracts from consumer branches that the selectors would not pick, but that have not been verified successfully yet. With `includeWipPactsSince`, the provider verifies them too (as pending), so consumer feature branches get feedback without the provider configuring their branch names.

**In the project:** `includeWipPactsSince = "2026-01-01"`; the `feature/discounts` pact is verified as WIP.

### Q40. How are verification results published, and why?

With `pact.verifier.publishResults=true` and the provider's version and branch. The broker needs them to answer `can-i-deploy`; without published results nothing can be deployed safely.

**In the project:** Maven maps `PACT_PUBLISH_RESULTS`, `PROVIDER_VERSION`, `PROVIDER_BRANCH`; only the workflow scripts and CI publish, never a developer's ad-hoc run.

### Q41. Should every local provider test run publish results?

No: only CI builds of committed code, with a real version. A developer's local run could publish results for code that never exists in git and confuse `can-i-deploy`.

**In the project:** publishing is off unless the scripts set it.

### Q42. How do you verify from a local file instead of the broker?

`@PactFolder` (or `@PactUrl`). Good for a quick loop or a first setup. The two modes should not run together.

**In the project:** `ProductPactFolderTest` (disabled when a broker URL is set) and `ProductPactBrokerTest` (enabled only then).

### Q43. What does a failing verification look like?

The interaction, the provider state, and a diff of what was expected and what came back, per field. For example: `body: $ Actual map is missing the following keys: available`.

**In the project:** two mutation checks in the README's "Results".

### Q44. What if provider states need a real database?

Use a test database (Testcontainers or an in-memory one) and let state handlers insert rows, or call the service layer to create the data. Clean up per interaction. Avoid depending on shared environments.

**In the project:** an in-memory repository, so states call the repository directly.

---

## 5. Pact Broker and deployment safety

### Q45. What does the Pact Broker do?

Stores pacts with consumer versions and branches, verification results with provider versions and branches, environments and deployments, and answers questions about them: which pacts to verify (selectors, pending, WIP), and "can this version be deployed there?" (`can-i-deploy`). It also triggers webhooks and has a UI.

**In the project:** self-hosted in Docker Compose, `http://localhost:9292`.

### Q46. Open-source broker or PactFlow?

The open-source broker is free and self-hosted. PactFlow is the hosted, commercial version with authentication/teams, a nicer UI, and bi-directional contract testing (OpenAPI vs pacts).

**In the project:** the open-source broker (no account needed).

### Q47. What is a pacticipant version, and what should it be?

The version of a consumer or provider build that a pact or verification belongs to. It must be unique per build and traceable to the code: the git commit is the usual choice. Never a fixed "1.0.0" reused by many builds.

**In the project:** the short git commit (marked `+local` for uncommitted changes).

### Q48. Branches, tags and environments: what is the difference?

Branches record where a version was built (main, feature/x). Environments and deployments record where it runs (production, test). Tags were the older way to express both and are replaced by branches and environments.

**In the project:** `--branch` on publish, `PROVIDER_BRANCH` on verification, `record-deployment` to `production`.

### Q49. What is the matrix?

The table of every consumer version against every provider version with the verification result. `can-i-deploy` queries it.

**In the project:** `docs/images/broker-matrix.png`; `scripts/summary.mjs` prints it on the CI run page.

### Q50. How does can-i-deploy work?

For a pacticipant version and a target environment, the broker looks up the versions of all its integration partners deployed in that environment and checks the matrix: is there a successful verification for each pair? Exit code 0 means yes; anything missing or failed means no.

**In the project:** `pact-broker can-i-deploy --pacticipant checkout-web --version <sha> --to-environment production`.

### Q51. Why record deployments?

So the broker knows what runs where. The next `can-i-deploy` for a partner checks against the recorded version, and provider builds verify the deployed consumer versions (`deployedOrReleased` selector).

**In the project:** `record-deployment` after each successful `can-i-deploy`.

### Q52. record-deployment or record-release?

Deployment: one version replaces the previous one in the environment (a web service). Release: several versions are in use at the same time until support ends (mobile apps, libraries); use `record-release` and `record-support-ended`.

**In the project:** deployments (both are services).

### Q53. In which order do you deploy consumer and provider the first time?

Provider first: no consumer is in production yet, so nothing can break. Then the consumer, whose contract was verified against the provider version now in production.

**In the project:** `contract-flow.mjs` steps 5 and 6.

### Q54. What is can-i-merge?

Like `can-i-deploy`, but against the main branches of the integration partners instead of an environment: "can this feature branch be merged without breaking main?"

**In the project:** not used; the CLI supports it (`pact-broker can-i-merge`).

### Q55. What webhooks does a broker offer, and which one matters most?

Webhooks call a URL on events: `contract_content_changed`, `contract_requiring_verification_published`, `provider_verification_published` and others. The key one: `contract_requiring_verification_published` starts the provider's verification pipeline for exactly the new contract, so the consumer gets an answer quickly.

**In the project:** described in the README and design doc; not configured (one repository, one pipeline).

### Q56. What if can-i-deploy runs while a verification is still running?

The result is "unknown" and the answer is no. `--retry-while-unknown <times>` with `--retry-interval` waits for the result (typically after a webhook-triggered verification).

**In the project:** not needed: the scripts verify before asking.

### Q57. How do you secure a broker?

Basic auth or bearer tokens (`PACT_BROKER_BASIC_AUTH_*`), read-only and read-write users, HTTPS, network restrictions. Never commit credentials; CI reads them from secrets (`PACT_BROKER_TOKEN`).

**In the project:** an open local broker (documented as a limitation).

### Q58. What happens to old data?

The broker can clean old versions on a schedule (`PACT_BROKER_DATABASE_CLEAN_*`), keeping what is deployed or on main.

**In the project:** a fresh broker per CI run; `npm run broker:down` deletes local data.

### Q59. Why did the example data cause a bug in this project?

The broker seeds demo data ("Example App") with `production` and `test` environments on first start. After switching the seeding off, the environments were gone and `can-i-deploy` failed with "Environment does not exist", and the breaking demo counted that failure as the expected "no". Fix: create the environment when missing, and check in the matrix that the "no" comes from a failed verification.

**In the project:** `ensureEnvironment()` and `verificationResult()` in `scripts/lib.mjs`.

### Q60. How do you see why can-i-deploy said no?

Its output lists each pair with the result and a reason ("the verification ... failed", "no verification result", "no version in environment"). The matrix and the verification result's diff show the details.

**In the project:** `docs/images/can-i-deploy.png`.

---

## 6. CI/CD and team process

### Q61. What does the consumer's pipeline do?

Unit + contract tests → publish the pact with version and branch → ( `can-i-deploy` → deploy → `record-deployment` ). Publishing happens on every branch, so providers see work in progress.

**In the project:** consumer job + steps 2, 3, 6 of `contract-flow.mjs` .

### Q62. What does the provider's pipeline do?

Own tests → verify the pacts selected by the broker, publish results → ( `can-i-deploy` → deploy → `record-deployment` ). Plus a webhook-triggered job that verifies a single changed pact.

**In the project:** provider job + steps 4 and 5 of `contract-flow.mjs` .

### Q63. Who owns the contract?

The consumer team writes it; the provider team owns making it pass and keeping the provider states. Changes are a conversation: the pact (and the broker's pending/WIP features) makes that conversation concrete.

**In the project:** the two sides live in separate folders with separate build tools, like two teams.

### Q64. A consumer needs a new field. What is the process?

The consumer adds the expectation on a feature branch and publishes; the provider sees it as a WIP pact (failing, but not breaking its build); the provider implements the field and its verification passes; then the consumer's `can-i-deploy` says yes and it can merge and deploy.

**In the project:** the first half is the breaking-change demo.

### Q65. The provider wants to remove a field. How do they know if it is safe?

Remove it on a branch and run the verification: if any consumer's contract (main or deployed versions) uses the field, verification fails with the consumer's name. If nobody uses it, it passes, and the removal is safe.

**In the project:** the mutation check (renaming `available` ) shows the failure the provider would see.

### Q66. How do you run all of this in one repository for a demo?

A script that runs the steps in order with one version per commit, and a broker in Docker. In a real system each service has its own repository and pipeline.

**In the project:** `npm run contract` , `npm run demo:breaking` , `.github/workflows/contract.yml` .

### Q67. How does the broker start in CI?

As Docker Compose (same file as locally) with health checks, or as service containers. Compose keeps one definition for both and handles the database dependency.

**In the project:** `npm run broker:up` inside the contract job.

---

## 7. Scenarios and pitfalls

### Q68. Common mistakes in consumer tests?

Exact values instead of matchers; asserting fields the consumer does not use; testing the test (calling `fetch` instead of the client); one giant interaction; provider states that describe the consumer's steps instead of the provider's data.

**In the project:** matchers everywhere, real client, one interaction per behaviour, data-oriented states.

### Q69. Common mistakes on the provider side?

Mocking the provider's own logic; state handlers that depend on order or leave data behind; verifying a stale local file instead of the broker in CI; publishing results from local runs; forgetting that a shared Spring context carries state between test classes.

**In the project:** the shared-context issue happened: `ProductApiTest` failed after the Pact states changed the catalogue, fixed with a reseed per test.

### Q70. The consumer test passes but verification fails. What do you check?

The diff first. Then: is the provider state implemented and does it create the right data? Does a request filter change too much (or not enough)? Are formats the same (dates with fractions, decimal numbers, charset in Content-Type)? Is the consumer's expectation too strict (exact values)?

**In the project:** the ISO-instant regex allows Java's optional fraction; the Content-Type matcher allows a charset.

### Q71. How do you contract-test optional fields?

Pact has no "optional" matcher on purpose: an interaction is one concrete example. Write two interactions with two provider states (with and without the field) if the consumer handles both.

**In the project:** not needed; every field used is always present.

### Q72. How do you deal with authentication that the provider cannot fake?

Use a request filter with a real test token, or configure the provider's test profile to accept a test token, or stub the identity provider. The contract keeps a matcher, not a real token.

**In the project:** a filter that injects `dev-token-123`, which the provider accepts via `app.api-tokens`.

### Q73. Can Pact test GraphQL?

Yes: GraphQL over HTTP is a POST with a query; Pact JS has a GraphQL interaction helper. The contract is still per query the consumer sends.

**In the project:** not used (REST).

### Q74. What does contract testing not replace?

Provider functional tests, a few end-to-end smoke tests of critical paths, performance tests, security tests, and checks of non-HTTP integration details (timeouts, retries, circuit breakers) in the consumer.

**In the project:** the provider keeps `ProductApiTest`; my portfolio has separate end-to-end (Cypress, Playwright) and performance (k6) projects.

### Q75. Describe this project in two minutes.

"Two services in two languages: a TypeScript checkout client and a Java Spring Boot product API. The consumer's Pact tests run its real HTTP client against Pact's mock server and write six interactions, including 401, 404 and 409, with matchers and parameterised provider states. The provider verifies them with JUnit 5 against the real Spring Boot app, with a request filter for the token. Contracts and results go through a self-hosted Pact Broker; the pipeline runs can-i-deploy and records deployments. A demo shows a consumer feature branch whose contract the provider does not meet yet: the provider build stays green thanks to pending pacts, while can-i-deploy blocks the consumer. Everything runs with one command and in GitHub Actions."

**In the project:** README.

### Q76. What would you add next?

A second consumer, message pacts for an event, a broker webhook to a provider pipeline, `can-i-merge` on pull requests, broker authentication, and a provider with a real database via Testcontainers.

**In the project:** "Known limitations" in the README and design doc section 7.

---

## 8. Quick reference

| Need | Command or code |
|---|---|
| consumer interaction | `pact.addInteraction().given(state, params).uponReceiving(desc).withRequest(...).willRespondWith(...).executeTest(...)` |
| type matcher | `integer(25)`, `decimal(19.99)`, `string('x')`, `like({...})` |
| format matcher | `regex(/^[A-Z]{3}$/, 'EUR')`, `uuid()`, ISO date/time regex |
| array | `eachLike(template)` |
| provider test | `@Provider`, `@PactBroker` or `@PactFolder`, `@TestTemplate` + `PactVerificationInvocationContextProvider` |
| provider state | `@State("a product exists") void s(Map<String, Object> params)` |
| request filter | change the `HttpRequest` in the `@TestTemplate` method before `verifyInteraction()` |
| selectors | `new SelectorBuilder().mainBranch().deployedOrReleased().matchingBranch()` |
| pending / WIP | `enablePendingPacts = "true"`, `includeWipPactsSince = "<date>"` |
| publish | `pact-broker publish pacts --consumer-app-version <sha> --branch <branch>` |
| publish results | `pact.verifier.publishResults=true`, `pact.provider.version`, `pact.provider.branch` |
| environment | `pact-broker create-environment --name production --production` |
| deploy check | `pact-broker can-i-deploy --pacticipant <name> --version <sha> --to-environment production` |
| record | `pact-broker record-deployment --pacticipant <name> --version <sha> --environment production` |